Design Choices and Data Structures

Overview

The telephone directory is implemented using **a two-dimensional array of Record objects**, where each row represents an independent hash table and each column represents a slot in that table. The array consists of 9 hash tables, corresponding to the possible first digits (1–9) of a valid phone number, and each hash table contains 20 slots.

Record Structure

Each slot in a hash table stores a Record object containing:

- Name
- Phone Number
- Address
- Status (EMPTY, OCCUPIED, or DELETED)

The 'Status' field is used to keep track of the state of each slot.

- EMPTY indicates that the slot has never been used.
- OCCUPIED indicates that the slot currently contains a valid record.
- DELETED indicates that a record previously existed in the slot but has been deleted.

A deleted slot is not marked as EMPTY because doing so would break the probing sequence. By marking it as DELETED, future searches continue probing beyond that slot, ensuring that records inserted after a collision can still be found.

Table Selection

The first digit of the phone number determines which of the nine hash tables will store the record.

For eg:

- Phone number 1482910 is stored in Table 1.
- Phone number 5482910 is stored in Table 5.
- Phone number 9482910 is stored in Table 9.

Since Java arrays are indexed from 0, the table index is calculated as:

```
(first digit – '0') -1
```

Hash Function

After selecting the appropriate table, the insertion position is determined using the last six digits of the phone number.

The hash function is:

Hash = (Last Six Digits) mod 20

For eg, for the phone number 5482910:

- First digit = 5 -> Table 5
- Last six digits = 482910
- 482910 % 20 = 10

The record is therefore first checked or inserted at Table 5, Slot 10.

Collision Handling

Different phone numbers can generate the same hash value, resulting in a collision.

To handle collisions, **linear probing** is used. If the computed slot is already occupied, the algorithm checks the next slot one by one until an available slot is found. If the end of the table is reached, probing wraps around to the beginning using the modulo operator.

For eg:

Hash = 18

Slot 18 -> Occupied

Slot 19 -> Occupied

Slot 0 -> Empty

Insert at Slot 0

The same probing sequence is followed during insertion, searching, deletion, and modification so that all operations remain consistent.

Phone Number Validation

Before performing any operation, the phone number is validated to ensure that:

- it is not null,
- it contains exactly seven digits,
- all characters are numeric,
- the first digit is between 1 and 9.

Validation is performed in the insert(), delete(), and modify() operations before hashing or table selection. This prevents invalid inputs from reaching the helper methods and avoids runtime errors.

Insert Operation

The insertion process follows these steps:

1. Validate the phone number.
2. Determine the correct hash table using the first digit.
3. Compute the hash value using the last six digits.
4. Search for duplicate phone number.
5. Perform linear probing until an EMPTY or DELETED slot is found.
6. Insert the new record and mark the slot as OCCUPIED.

If all 20 slots in the selected table are occupied, the insertion fails and the table is reported as full.

Search Operation

The search operation follows the same probing sequence used during insertion.

Starting from the computed hash value, each slot is examined until:

- the required phone number is found,
- an EMPTY slot is encountered, indicating that the record does not exist, or
- all 20 slots have been checked.

DELETED slots are skipped because the desired record may still exist further along the probing sequence.

The same search method is reused by the insert, delete, and modify operations, reducing code duplication.

Delete Operation

Deletion first validates the phone number and locates the corresponding record using the same search procedure used during insertion.

If the record is found, its status is changed from OCCUPIED to DELETED instead of removing the record completely.

Using a DELETED marker preserves the probing sequence, ensuring that subsequent searches continue correctly even after records have been removed.

Modify Operation

The implementation supports two types of modification.

Address Modification

If only the address is changed, the phone number remains the same. Therefore, the record stays in its current position, and only the address field is updated.

Phone Number Modification

If the phone number changes, the record may belong to a different table or hash position.

The implementation first validates the new phone number and checks that it does not already exist. If the new phone number is valid and unique, the old record is deleted and a new record containing the updated phone number and address is inserted.

This ensures that the record is placed in the correct table and follows the same hashing and collision-resolution strategy as every other record.

Overall Design

The implementation is divided into small helper methods for tasks such as table selection, hashing, searching, and validation. This keeps the code modular, avoids repeating the same logic in multiple places, and makes individual operations easier to understand and maintain.

All operations use the same hashing and linear probing strategy, ensuring consistent behaviour throughout the directory. The use of DELETED markers allows the hash tables to continue functioning correctly even after multiple insertions and deletions.